Sesión 14 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Tipos de datos y funciones asociadas

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Tipos sencillos

Ejercicio 1.

Define `data Fecha = CFecha Int Int Int` (día, mes y año) y `esAnterior :: Fecha -> Fecha -> Bool` que compruebe si la primera fecha es estrictamente anterior a la segunda (comparando primero el año, luego el mes, y por último el día).

```
ghci> esAnterior (CFecha 1 1 2024) (CFecha 1 1 2025)
True
ghci> esAnterior (CFecha 15 6 2024) (CFecha 3 6 2024)
False
```

Ejercicio 2. ★

Define `data Vector2D = CVector2D Double Double` junto con `sumaVectores :: Vector2D -> Vector2D -> Vector2D`, `escala :: Double -> Vector2D -> Vector2D` y `modulo :: Vector2D -> Double`.

```
ghci> sumaVectores (CVector2D 1 2) (CVector2D 3 4)
CVector2D 4.0 6.0
ghci> modulo (CVector2D 3 4)
5.0
```

Ejercicio 3. ★

Define `data Libro = CLibro String Int Bool` (título, año de publicación y si está prestado o no) junto con `disponible :: Libro -> Bool`, que indique si el libro se puede prestar, y `prestar :: Libro -> Libro`, que devuelva el mismo libro marcado como prestado.

```
ghci> disponible (CLibro "Dune" 1965 False)
True
ghci> prestar (CLibro "Dune" 1965 False)
CLibro "Dune" 1965 True
```

Parte B — Tipos con `deriving`

Ejercicio 4.

Define `data Moneda = Euro | Dolar | Libra deriving (Eq, Show)`, una función `tasaCambio :: Moneda -> Double` (el valor de una unidad de esa moneda en euros, con las tasas que prefieras) y `convertir :: Moneda -> Moneda -> Double -> Double` que convierta una cantidad de una moneda a otra.

```
ghci> convertir Dolar Euro 100
92.0
```

Ejercicio 5. ★

Define `data Palo = Picas | Corazones | Diamantes | Treboles deriving (Eq, Show)` y `data Carta = CCarta Int Palo deriving (Eq, Show)`. Escribe a mano `instance Ord Carta` de forma que las cartas se ordenen únicamente por su valor numérico, **ignorando** el palo.

```
ghci> CCarta 5 Picas < CCarta 7 Corazones
True
ghci> compare (CCarta 5 Picas) (CCarta 5 Corazones)
EQ
```

Parte C — `Maybe` con tipos propios

Ejercicio 6.

Reutilizando `Fecha` de la Parte A, define `buscarPorId :: Int -> [(Int, Fecha)] -> Maybe Fecha` que busque la fecha asociada a un identificador en una lista de pares.

```
ghci> buscarPorId 2 [(1, CFecha 1 1 2020), (2, CFecha 5 5 2021)]
Just (CFecha 5 5 2021)
ghci> buscarPorId 9 [(1, CFecha 1 1 2020)]
Nothing
```

Ejercicio 7.

Reutilizando `Vector2D`, define `combinarVectores :: Maybe Vector2D -> Maybe Vector2D -> Maybe Vector2D` que sume dos vectores **solo** si ambos existen (es decir, si ambos son `Just`), devolviendo `Nothing` en caso contrario.

```
ghci> combinarVectores (Just (CVector2D 1 2)) (Just (CVector2D 3 4))
Just (CVector2D 4.0 6.0)
ghci> combinarVectores Nothing (Just (CVector2D 3 4))
Nothing
```

Parte D — Árbol con forma distinta: sistema de archivos

Ejercicio 8.

Define un tipo para representar un sistema de archivos sencillo:

```
data Entrada = Archivo String Int
             | Carpeta String [Entrada]
```

donde `Archivo nombre tamano` representa un fichero, y `Carpeta nombre contenido` representa un directorio con una lista de entradas dentro (archivos y/o subcarpetas). Define `tamanoTotal :: Entrada -> Int` que calcule el tamaño total, sumando recursivamente el contenido de las carpetas.

```
ghci> sistema = Carpeta "raiz"
        [Archivo "a.txt" 100,
         Carpeta "fotos" [Archivo "x.jpg" 500, Archivo "y.jpg" 300]]
ghci> tamanoTotal sistema
900
```

Ejercicio 9.

Define `contarArchivos :: Entrada -> Int` que cuente cuántos archivos (no carpetas) hay en total, recursivamente.

```
ghci> contarArchivos sistema
3
```

Ejercicio 10.

Define `buscarArchivo :: String -> Entrada -> Maybe Int` que busque un archivo por su nombre en todo el árbol (incluyendo subcarpetas) y devuelva su tamaño si lo encuentra.

```
ghci> buscarArchivo "x.jpg" sistema
Just 500
ghci> buscarArchivo "no_existe.txt" sistema
Nothing
```

Parte E — Expresiones booleanas

Ejercicio 11. ★

Define un tipo para representar expresiones lógicas:

```
data BoolExpr = BVal Bool
              | BAnd BoolExpr BoolExpr
              | BOr  BoolExpr BoolExpr
              | BNot BoolExpr
```

y `evalB :: BoolExpr -> Bool` que la evalúe.

```
ghci> evalB (BAnd (BVal True) (BNot (BVal False)))
True
```

Ejercicio 12.

Define `simplificar :: BoolExpr -> BoolExpr` que aplique algunas reglas de simplificación sencillas, sin evaluar la expresión por completo: `BAnd (BVal False) _` y `BAnd _ (BVal False)` se simplifican a `BVal False`; `BOr (BVal True) _` y `BOr _ (BVal True)` se simplifican a `BVal True`; en cualquier otro caso, la expresión se devuelve igual (sin simplificar sus subexpresiones).

```
ghci> simplificar (BAnd (BVal False) (BVal True))
BVal False
ghci> simplificar (BAnd (BVal True) (BVal True))
BAnd (BVal True) (BVal True)
```

Parte F — Ejercicio integrador: mini inventario

Ejercicio 13. ★

Define `data Producto = CProducto String Double Int` (nombre, precio y stock disponible) y `valorTotalInventario :: [Producto] -> Double` que calcule la suma de `precio * stock` de todos los productos.

```
ghci> inventario = [CProducto "Teclado" 25 10, CProducto "Raton" 15 20]
ghci> valorTotalInventario inventario
550.0
```

Ejercicio 14. ★

Define `buscarProducto :: String -> [Producto] -> Maybe Producto` y `aplicarDescuento :: Double -> Producto -> Producto` (donde el argumento `Double` es el porcentaje de descuento, por ejemplo `10` para un 10 %).

```
ghci> aplicarDescuento 10 (CProducto "Teclado" 25 10)
CProducto "Teclado" 22.5 10
```

Ejercicio 15. ★

Combina las piezas anteriores: define `calcularPrecioFinal :: String -> Double -> [Producto] -> Either String Double` que, dado el nombre de un producto, un porcentaje de descuento, y el inventario, devuelva (`Right precio`) el precio final con descuento si el producto existe y tiene stock disponible, o (`Left motivo`) un mensaje explicando por qué no se puede calcular si el producto no existe o no tiene stock.

```
ghci> calcularPrecioFinal "Teclado" 10 inventario
Right 22.5
ghci> calcularPrecioFinal "Monitor" 10 inventario
Left "Producto no encontrado"
```

Pista: `Either a b` es como `Maybe b`, pero con un valor asociado también al caso de fallo: `data Either a b = Left a | Right b`.